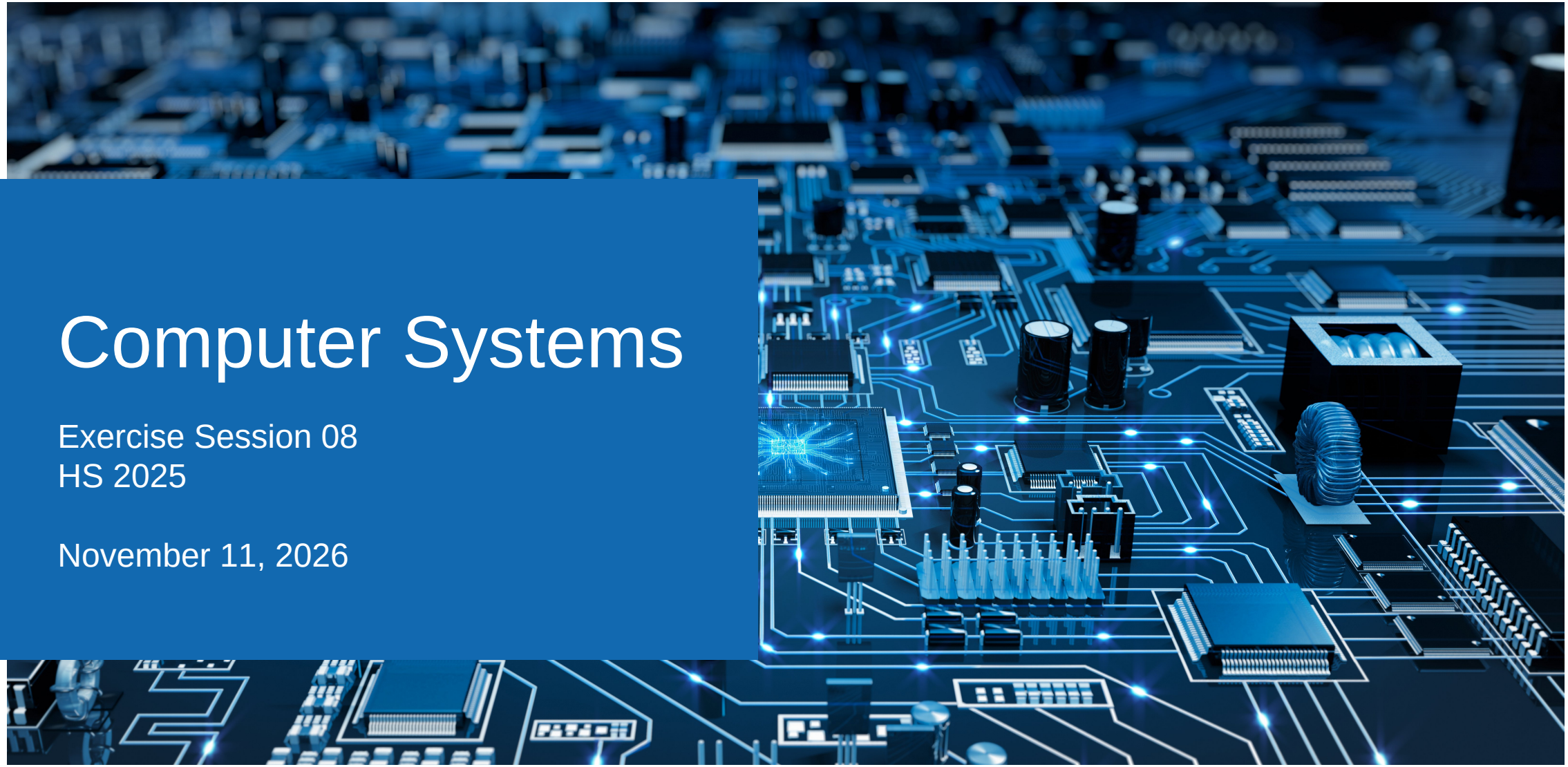
ETH zürich

Systems@ETH zürich



Computer Systems

Exercise Session 08
HS 2025

November 11, 2026

Distributed Systems ?

Systems@ETH zürich

- 1. Scalability
  - More requests, more data, more ... ==> **Need many machines to act as one**
    - One database split across many different servers
    - Different servers serving the same website
    - ...
- 2. Fault tolerance
  - Ensure uptime given failure of servers
    - Multiple replicas of database
    - Ensure synchronization between multiple copies to prevent drift

Agenda

Systems@ETH zürich

- Why do we need logical time?
- Happens-Before Relation
- Lamport's Logical Clocks
  - Totally-Ordered Multicast
- Vector Clocks
  - Causally-Ordered Multicast
- Quiz

Why do we need logical time?

Systems@ETH zürich

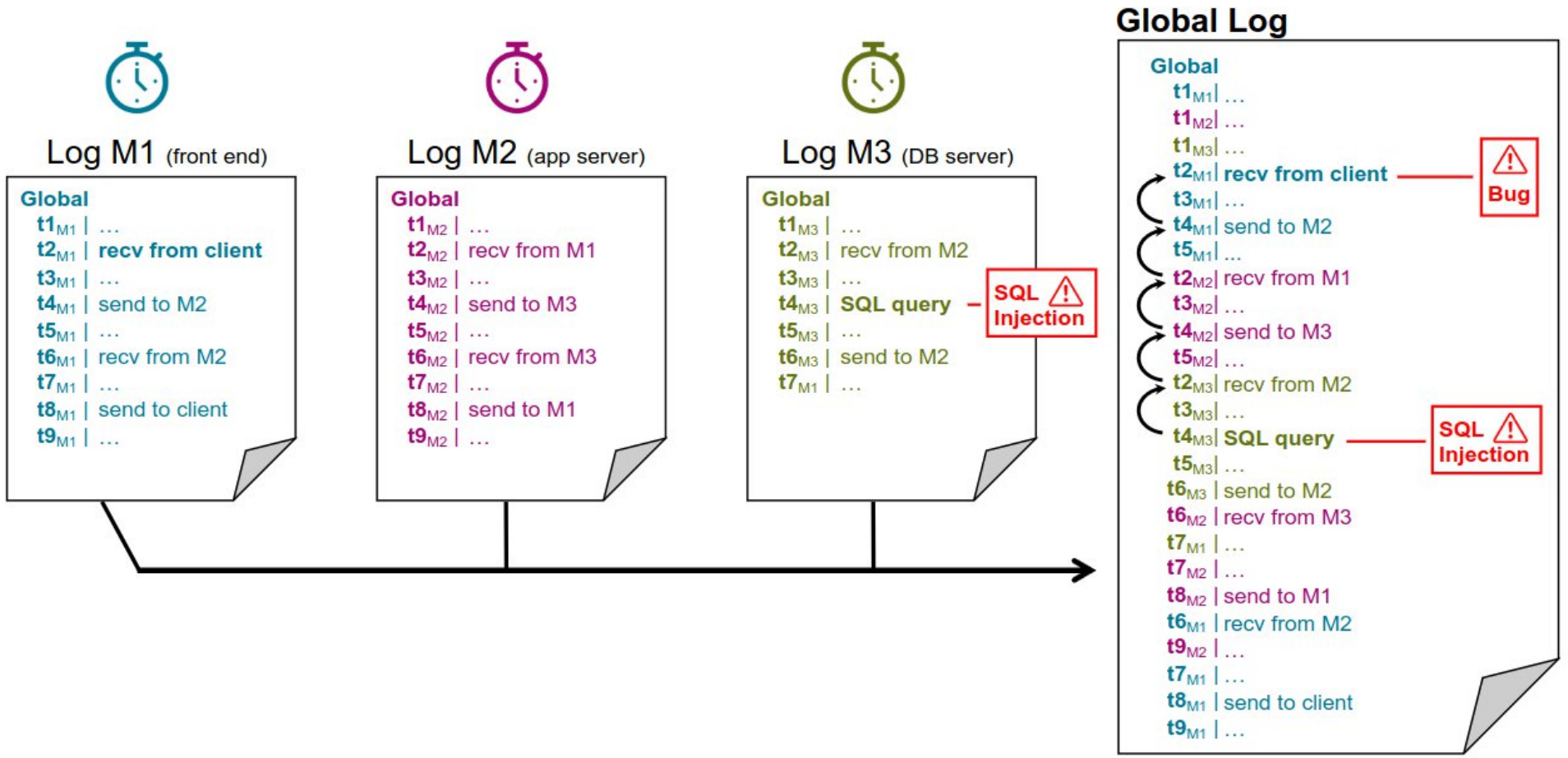
In **real distributed systems** there is:

- Network delay
- Clock drift
- (Partial) Failures

→ no exact synchronization (e.g. NTP)

Use logical time for *causality* and *correct ordering*.

Event Ordering



Time on a single Machine

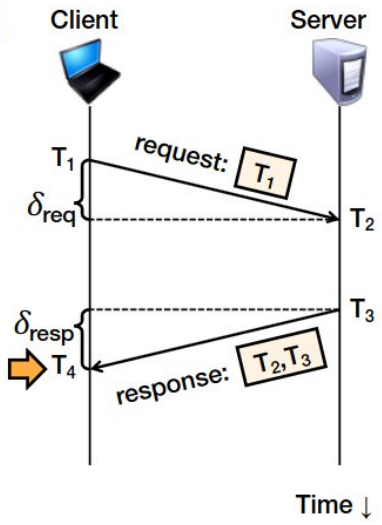
```
-- Boot 7cd59c1f1adc4e4788f00a2fc8b1dcb4 --
Oct 19 07:28:04 thinkpad systemd-timesyncd[485]: Network configuration changed, trying to establish connection.
Oct 19 07:28:05 thinkpad systemd-timesyncd[485]: Network configuration changed, trying to establish connection.
Oct 19 07:28:08 thinkpad systemd-timesyncd[485]: Network configuration changed, trying to establish connection.
Oct 19 07:28:09 thinkpad systemd-timesyncd[485]: Network configuration changed, trying to establish connection.
Oct 19 07:28:09 thinkpad systemd-timesyncd[485]: Network configuration changed, trying to establish connection.
Oct 19 07:28:09 thinkpad systemd-timesyncd[485]: Network configuration changed, trying to establish connection.
Oct 19 07:28:44 thinkpad systemd-timesyncd[485]: Contacted time server 91.189.91.157:123 (ntp.ubuntu.com).
Oct 19 07:28:44 thinkpad systemd-timesyncd[485]: Initial clock synchronization to Mon 2026-10-19 07:28:44.416450 CEST.
Oct 19 15:25:14 thinkpad systemd-timesyncd[485]: Contacted time server 91.189.91.157:123 (ntp.ubuntu.com).
Oct 19 15:57:31 thinkpad systemd-timesyncd[485]: Contacted time server 185.125.190.57:123 (ntp.ubuntu.com).
Oct 19 22:42:40 thinkpad systemd-timesyncd[485]: Contacted time server 185.125.190.56:123 (ntp.ubuntu.com).
Oct 20 13:26:49 thinkpad systemd-timesyncd[485]: Contacted time server 185.125.190.58:123 (ntp.ubuntu.com).
Oct 20 15:57:02 thinkpad systemd-timesyncd[485]: Contacted time server 185.125.190.56:123 (ntp.ubuntu.com).
```

Goal: Client sets clock ← $T_3 + \delta_{resp}$

- Client samples **round trip time (δ)**
  $\delta = \delta_{req} + \delta_{resp} = (T_4 - T_1) - (T_3 - T_2)$
- But client knows δ, not δ_{resp}

Assume: $\delta_{req} \approx \delta_{resp}$

Client sets clock ← $T_3 + \frac{1}{2}\delta$



Time on multiple machines

Systems@ETH zürich

– Perfectly synchronizing clocks very difficult
– A => B could lead do time(B) < time(A)

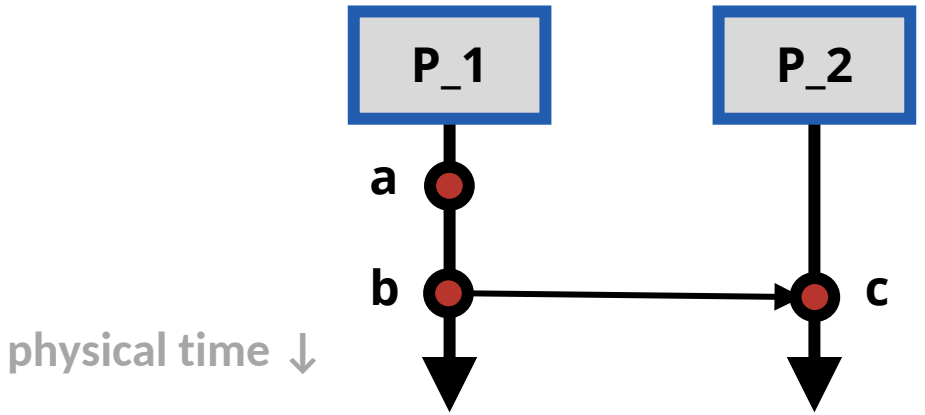
Often we are not interested in the absolute times,

but only in temporal relationship between events

Happens-Before Relation

- **Notation:**
  Event a **happens before** event b: **"a → b"**

- **Rules:**
  Program Order: If **a** before **b** in *same process*, then **a → b**.
  Message: If **a** is *send(m)* and **b** is *receive(m)*, then **a → b**.
  Transitivity: If a → b and b → c, then **a → c**.



Lamport's Logical Clocks

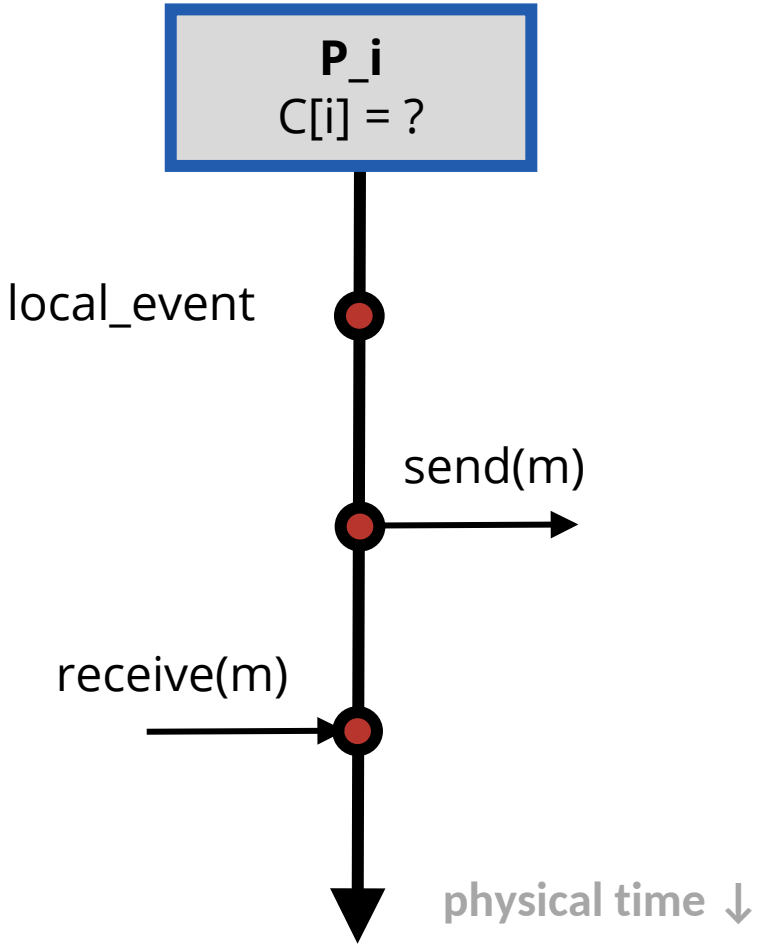
- **Rules:**

```
init():
    C[i] = 0

local_event():
    C[i] = C[i] + 1

send(m):
    C[i] = C[i] + 1
    m.timestamp = C[i]
    transmit(m)

receive(m):
    C[i] = max(C[i], m.timestamp) + 1
```



Lamport's Logical Clocks

The ETH Zürich library runs a small distributed system. Three processes participate: *Hönggerberg* (P_H), *Zentrum* (P_Z), and *Catalog* (P_C).

We see the following events (in process order):
P_H: **(a)** *student returns book B*; **(b)** *send return notice to Catalog*.
P_Z: **(c)** *students requests to borrow book B*; **(d)** *send borrow request to Catalog*.
P_C: **(e)** *local audit check*; **(f)** *receive return notice*; **(g)** *receive borrow request*.

Task 1.1: Assign Lamport timestamps to the seven events (a-g) above.
Task 1.2: Produce a deterministic total order of all events. Break ties if necessary.

Lamport's Logical Clocks

Systems@ETH Zürich

The ETH Zürich library runs a small distributed system. Three processes participate: *Hönggerberg* (P_H), *Zentrum* (P_Z), and *Catalog* (P_C).

We see the following events (in process order):
P_H: *(a) student returns book B; (b) send return notice to Catalog.*
P_Z: *(c) students requests to borrow book B; (d) send borrow request to Catalog.*
P_C: *(e) local audit check; (f) receive return notice; (g) receive borrow request.*

Task 1.1: Assign Lamport timestamps to the seven events (*a*-*g*) above.
{a:1, b:2, c:1, d:2, e:1, f:3, g:4}
Task 1.2: Produce a deterministic total order of all events. Break ties if necessary.
Order by timestamp, break ties using **C<H<Z**:
{e:(1,C), a:(1,H), c:(1,Z), b:(2,H), d:(2,Z), f:(3,C), g:(4,C)}

Totally-Ordered Multicast

- A New York-based bank wants to make its transaction ledger database resilient to whole-site failures
- **Replicate** the database, keep one copy in sf, one in nyc



Initial Balance: 1000$
San Francisco: Deposit 100$ New York: 1% Interest

Read => nearest copy, update => both copies

Totally-Ordered Multicast

Systems@ETH zürich

- **Rules:**

  On **event**: create Lamport timestamp, **broadcast update** to all (incl. self).

  On **update**: insert into local queue,

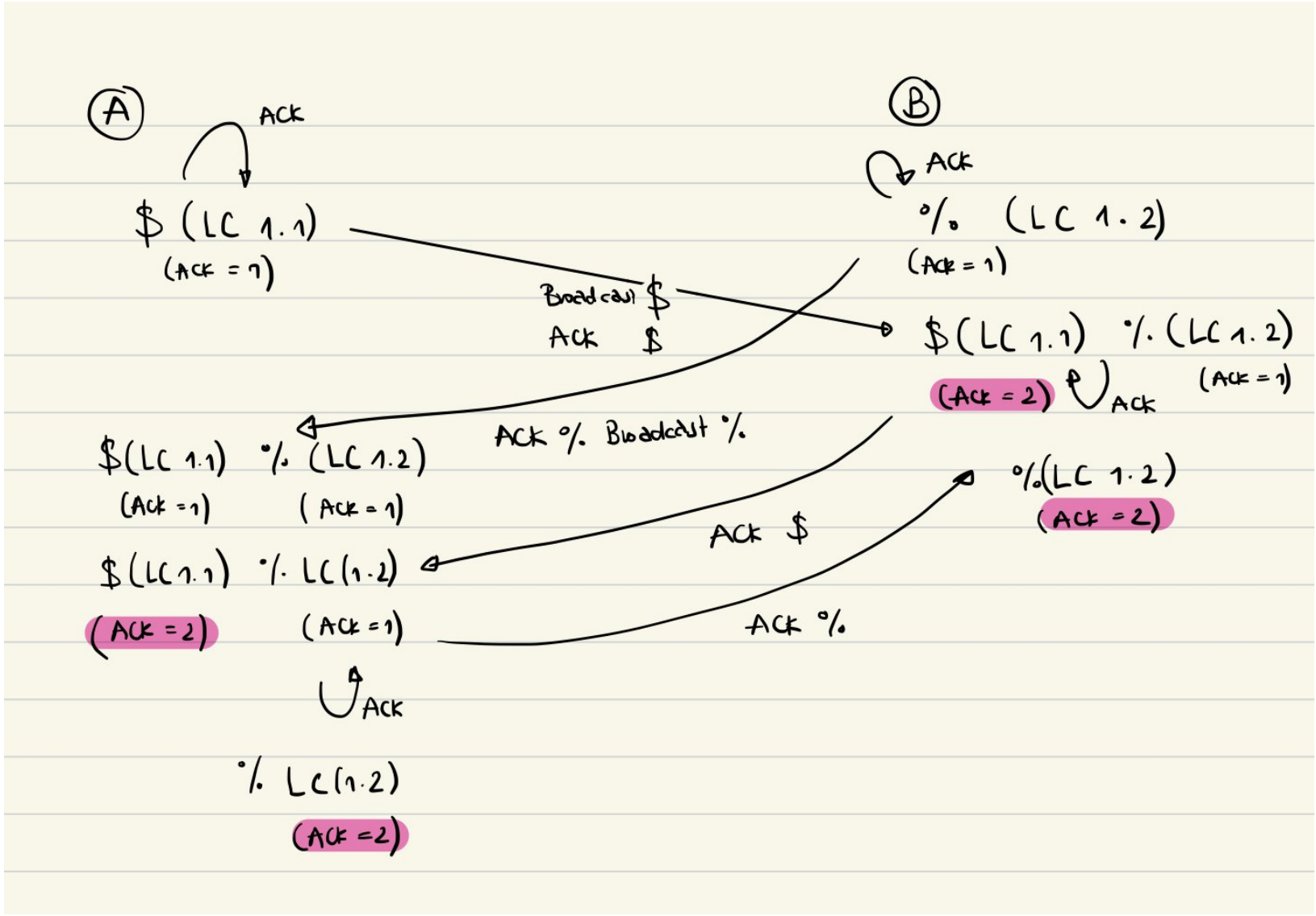
  send ack to all (incl. self) if inserted to head of queue.

  For head of queue, on **ack** from all:

  remove from queue

  process event.

  send ack to all (incl. Self) for the new head of queue


A
ACK
$ (LC 1.1)
(ACK = 1)
Broadcast $
ACK $
B
ACK
% (LC 1.2)
(ACK = 1)
$(LC 1.1) % (LC 1.2)
(ACK = 2)
ACK
(ACK = 1)
ACK % Broadcast %
$(LC 1.1) % (LC 1.2)
(ACK = 1) (ACK = 1)
$(LC 1.1) % LC(1.2)
(ACK = 2) (ACK = 1)
ACK
% LC(1.2)
(ACK = 2)
ACK $
% (LC 1.2)
(ACK = 2)
ACK %

Totally-Ordered Multicast

SBB runs replicated timetable servers in three locations:
Zürich (R_Z), *Olten* (R_O), and *Bern* (R_B).

We see the following events (in process order):
R_Z: **(a)** *local query*; **(b)** *operator issues update U for train 1*; [...]
R_B: **(c)** *local query*; **(d)** *local query*; ~~**(e)** *operator issues update V for train 2*; [...]~~

U is delivered in this order: R_Z, R_O, R_B.
Each receiver immediately sends ACK-U to all replicas.
~~ACK-U from R_Z and from R_O arrive at their destinations before V arrives at R_Z.~~
~~ACK-U from R_B is delivered right after R_Z receives V.~~

Totally-Ordered Multicast

Task 2.1: Compute the attached timestamps for updates U and V using TOM.

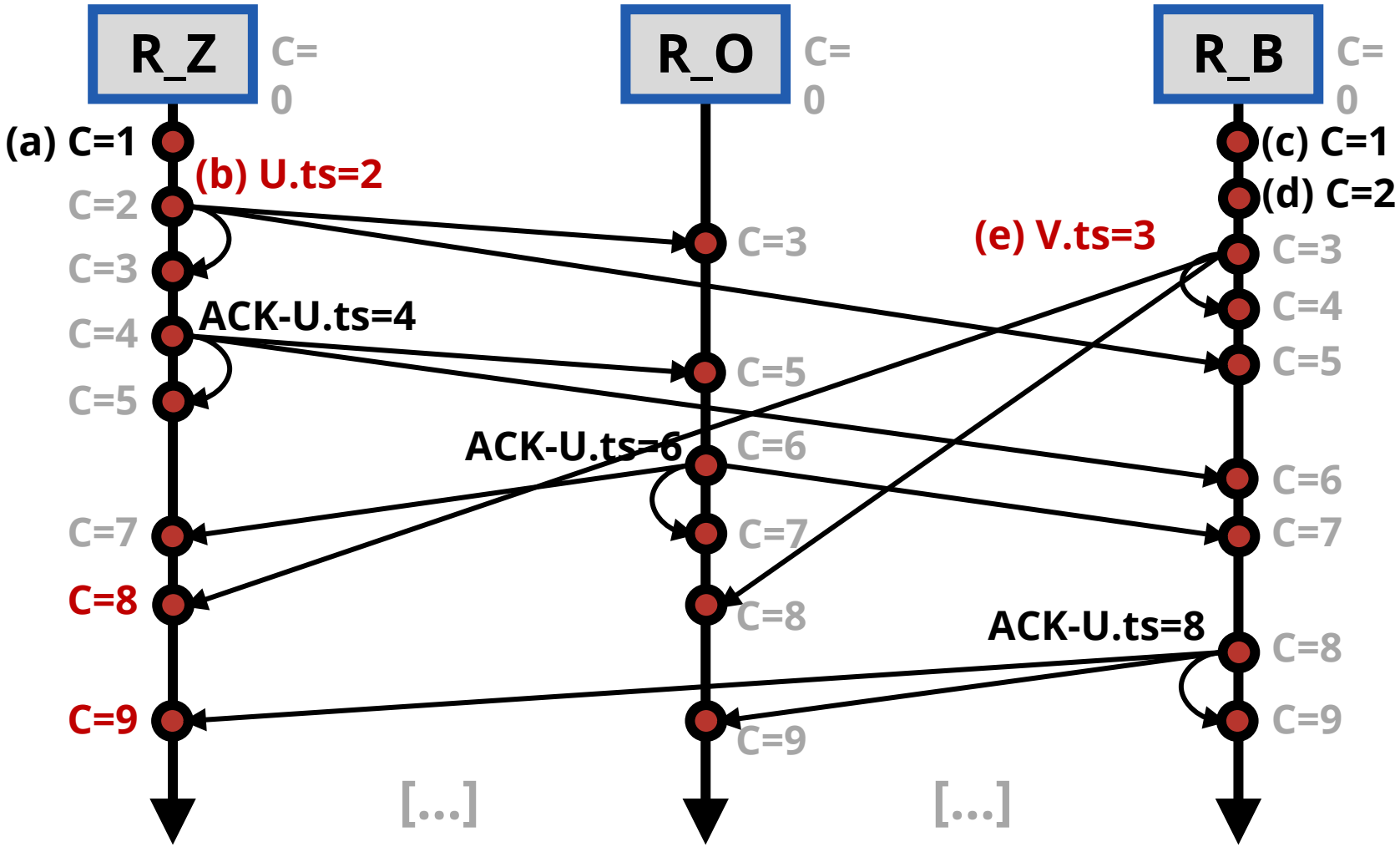
Task 2.2: What's the receive-time local clock at R_Z when it receives V?

Task 2.3: What's the receive-time local clock at R_Z when U gets applied?

Task 2.4: Is U → V true? Justify briefly.

Task 2.5: If R_O fails to ack, can R_Z apply U? Explain the implication.

Lamport's Logical Clocks



Totally-Ordered Multicast

Systems@ETH zürich

Task 2.1: Compute the attached timestamps for updates U and V.

{U:2, V:3}

Task 2.2: What's the receive-time local clock at R_Z when it gets V?

C_Z:8

Task 2.3: What's the receive-time local clock at R_Z when U gets applied?

C_Z:9

Totally-Ordered Multicast

Task 2.4: Is U → V true? Justify briefly.

Although timestamp(U) < timestamp(V), there is **no happens-before relation** U → V.
U and V were produced independently at different replicas without any message or causal chain between them.
Lamport's property guarantees (U → V) ⇒ timestamp(U) < timestamp(V), **but not the converse**.
Here timestamp(U) < timestamp(V) holds while U || V (concurrent).
TOM **serializes independent updates** (losing concurrency) because of timestamp ordering.

Totally-Ordered Multicast

Systems@ETH zürich

Task 2.5: If R_O fails to ack, can R_Z apply U? Explain the implication.

TOM requires *every replica's ack* before removing the queue head.
R_Z and R_B will **wait** for R_O's ack.
Until R_O's ack arrives, neither replica processes U nor V (they are both stuck behind the head), so **progress stalls**.
Availability implication: A single slow replica (R_O) can block the whole system
→ poor availability and poor scalability (due to all-to-all ack requirement)

Totally-Ordered Multicast

Task 2.6: Describe and implement a design that uses TOM to achieve mutual exclusion around a critical section.

See **lamport-cs.py**

Are We Done?

- *Does totally-ordered multicast solve the problem of multi-site replication in general?*
- Not by a long shot!
- Our protocol **assumed:**
  - —No node failures
  - —No message loss
  - —No message corruption
- All to all communication **does not scale**
- **Waits forever** for message delays **(performance?)**

Vector Clocks

Issue with Lamport Clocks:

a => b implies C(a) < C(b) BUT C(a) < C(b) could mean a ==> b or a || b

- **Rules:**

  At process i:

  Init: set [0,…,0]

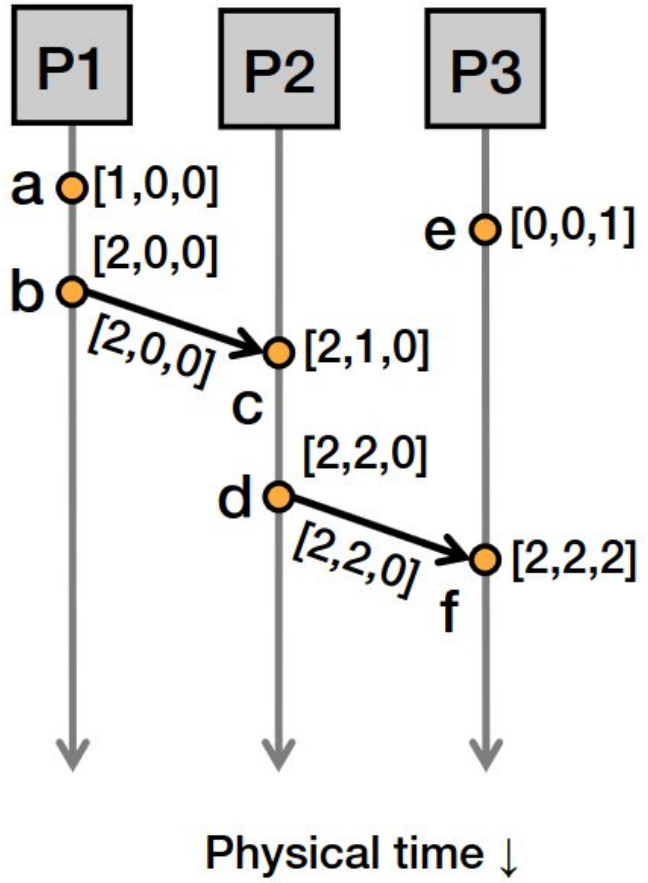
  Local event: increment own clock […,c_i+1,…]

  Receive message with ts [d_1,…,d_n]: 1. set each clock c_j = max(c_k,d_k)

  2. increment own clock […,c_i+1,…]

- All processes' VCs start at [0, 0, 0]

1. Applying local update rule
2. Applying message rule



Comparing Vector Timestamps

- Rule for comparing vector timestamps:
  - $V(a) = V(b)$ when $a_k = b_k$ for all k
    - They are the same event
  - $V(a) < V(b)$ when $a_k \leq b_k$ for all k and $V(a) \neq V(b)$
    - $a \rightarrow b$

- Concurrency:
  - $V(a) \parallel V(b)$ if $a_i < b_i$ and $a_j > b_j$, some i, j
    - $a \parallel b$

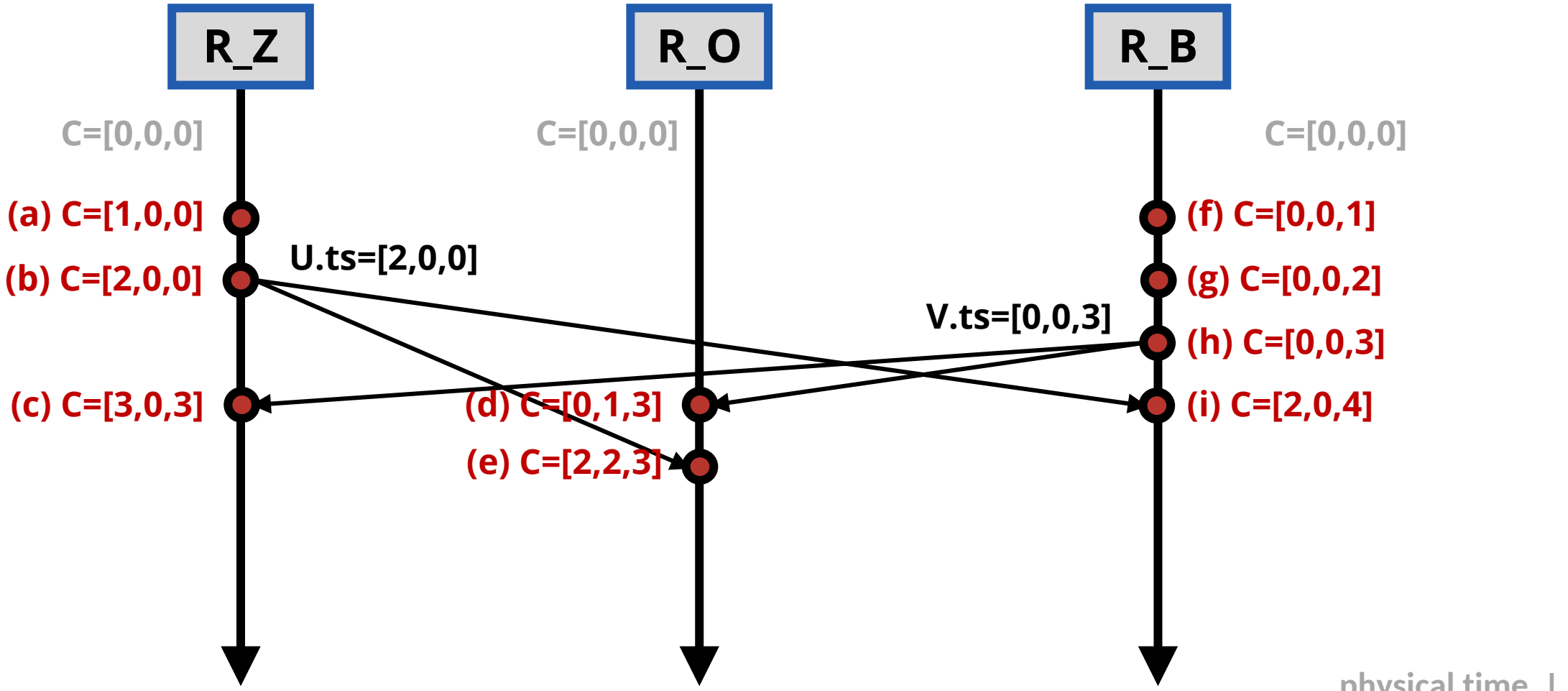
Vector Clocks

SBB runs replicated timetable servers in three locations:
Zürich (R_Z), *Olten* (R_O), and *Bern* (R_B).

We see the following events (in process order):
R_Z: **(a)** *local query*; **(b)** *issue update U for train 1*; **(c)** *receive V*.
R_O: **(d)** *receive V*; **(e)** *receive U*.
R_B: **(f)** *local query*; **(g)** *local query*; **(h)** *issue update V for train 2*; **(i)** *receive U*.

Task 3.1: Compute the vector timestamps for events (a-i).
Task 3.2: Are (b) and (h) causally related? Justify with vector comparison.
Task 3.3: Are (d) and (e) causally related? Justify with vector comparison.

Lamport's Logical Clocks



Vector Clocks

Task 3.1: Compute the vector timestamps for all events.

{a:[1,0,0], f:[0,0,1], g:[0,0,2], b:[2,0,0], h:[0,0,3], d:[0,1,3], c:[3,0,3], e:[2,2,3], i:[2,0,4]}

Task 3.2: Are (b) and (h) causally related? Justify with vector comparison.

No, **(b) and (h) are concurrent** (no happens-before relation).
Because neither [2,0,0]≤[0,0,3] nor [0,0,3]≤[2,0,0] (check component-wise).

Vector Clocks

Task 3.3: Are (d) and (e) causally related? Justify with vector comparison.

Yes, **d → e.**
Because [0,1,3]≤[2,2,3] (check component-wise).
Interpretation: This is a local causal ordering (R_O saw V before it saw U), not a global causal relation between the original sends.

Causally-Ordered Multicast

Systems@ETH zürich

- **Rules:**

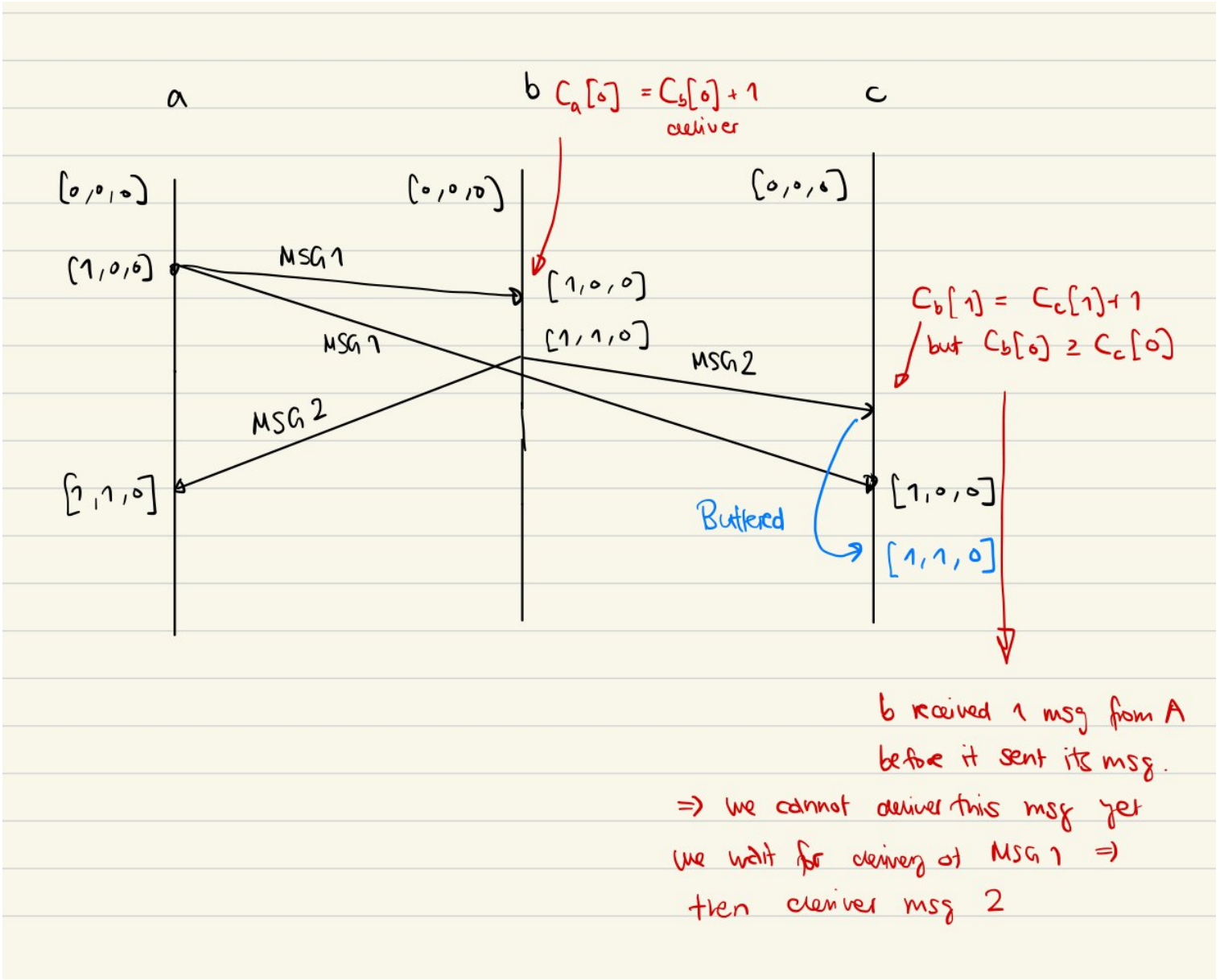
  On **event**: increment own vector clock, **broadcast update** to all (incl. self).

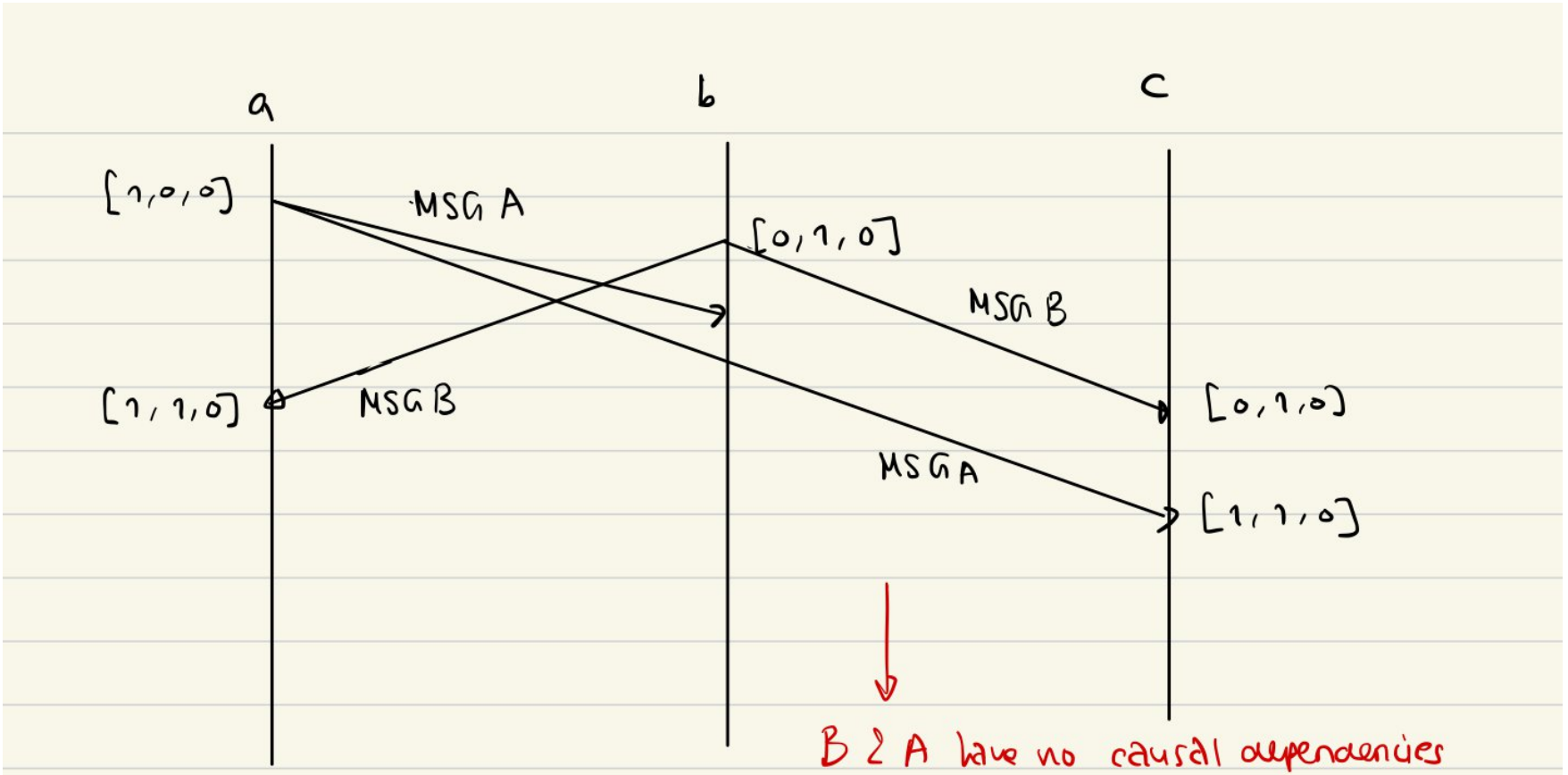
  On **receive**: insert into local queue,
  deliver update **if** timestamp's **deps are met** by local vector
  otherwise keep it buffered until deps are met

  Deliver update from sender s with timestamp M at receiver with local vector V **iff:**

  M[s] == V[s] + 1 *"next message from s in order"*

  for every k≠s: M[k] <= V[k] *"ensure at least same view as s"*


a
b $C_a[0] = C_b[0] + 1$
deliver
c
[0,0,0]
[0,0,0]
[0,0,0]
(1,0,0)
MSG 1
[1,0,0]
$C_b[1] = C_c[1] + 1$
but $C_b[0] \geq C_c[0]$
[1,1,0]
MSG 1
MSG 2
MSG 2
[1,1,0]
[1,0,0]
Buffered
[1,1,0]
b received 1 msg from A
before it sent its msg.
=> we cannot deliver this msg yet
we wait for delivery of MSG 1 =>
then deliver msg 2



B & A have no causal dependencies as when B, A were sent, both senders have not seen any other msg's.

⇒ At C, either A or B could be processed first.

Causally-Ordered Multicast

SBB runs replicated timetable servers in three locations:
Zürich (R_Z), *Olten* (R_O), and *Bern* (R_B).

We see the following events (in process order):
R_Z: **(a)** *local query*; **(b)** *issue update U for train 1*; **(c)** *receive V*.
R_O: **(d)** *receive V*; **(e)** *receive U*.
R_B: **(f)** *local query*; **(g)** *local query*; **(h)** *issue update V for train 2*; **(i)** *receive U*.

Task 3.4: Explain why replicas are not forced to serialize U and V under COM.
Task 3.5: State the advantages and disadvantages of COM over TOM.

Causally-Ordered Multicast

Task 3.4: Explain why replicas are not forced to serialize U and V under COM.

Vectors for U and V are incomparable.
COM delivers a message only when its causal dependencies are already seen.
Because U and V have no causal dependency, each replica may deliver U and V in any order (no buffering needed).
Short: COM only orders when causality exists; concurrent updates remain unconstrained.

Causally-Ordered Multicast

Task 3.5: State the advantages and disadvantages of COM over TOM.

Advantages:

- Preserves concurrency
- Better scalability & availability (less blocking)
- Semantically meaningful ordering

Disadvantages:

- Metadata & bookkeeping cost
- Non-determinism
- Stronger consistency (linearizability, consensus, etc.) requires extra mechanisms

Quiz

Systems@ETH zürich

1. Why can **clock synchronization** (NTP) be inadequate for reasoning about event ordering in distributed systems?
2. What does the **happens-before relation** (→) capture?
3. State the two **Lamport clock rules** for sending and receiving messages.
4. If **C(a) < C(b)** in Lamport timestamps, does that imply a → b?
5. How do you break ties to obtain a **total order** from Lamport timestamps?
6. Describe one shortcoming of **totally-ordered multicast**.
7. What is a **vector clock** and what benefit does it provide over Lamport clocks?
8. Describe a design that retains more **concurrency** than totally-ordered multicast.

Quiz

1. Why can **clock synchronization** (NTP) be inadequate for reasoning about event ordering in distributed systems?

Physical clocks cannot be perfectly synchronized (network delays, drift, failures). Comparing wall-clock timestamps can violate causality and produce incorrect event orders. Logical clocks reason about ordering without relying on synchronized physical time.

2. What does the **happens-before relation** (→) capture?

Ordering: local sequence, message exchange, and transitive closure of those.

Quiz

Systems@ETH zürich

3. State the two **Lamport clock rules** for sending and receiving messages.

Send: increment local clock (+1) and include the clock value in the message.
Receive: set local clock to max(localClock, timestampInMessage) + 1.

4. If **C(a) < C(b)** in Lamport timestamps, does that imply a → b?

No, Lamport timestamps preserve causality (a → b ⇒ C(a) < C(b)), but the converse is false: C(a) < C(b) does not guarantee a → b (they may be concurrent).

5. How do you break ties to obtain a **total order** from Lamport timestamps?

Append process identifier and compare pairs (ts, PID) lexicographically.

Quiz

6. Describe one shortcoming of **totally-ordered multicast**.

Serialization: Forces single global order even for concurrent independent events.
Availability/Scalability: blocks progress if it waits for acks from all replicas.

7. What is a **vector clock** and what benefit does it provide over Lamport clocks?

Array of counters (one per process).
Captures potential causality: it can distinguish concurrency from causal order (i.e., decide a → b, b → a, or a || b).

Quiz

8. Describe a design that retains more **concurrency** than totally-ordered multicast.

Causally-ordered multicast: using vector clocks, buffer a message until all its causal predecessors have been delivered.